Lab 6 Report: NLP Fundamentals – Tokenization, Embeddings & Text Chunking

CAI601400 – Artificial Intelligence Lab | Week 6 | CLO3 (C3: Apply) | Rasikh Ali – 42365

1. Objective and setup

The objective of this lab was to apply tokenization, text embeddings and chunking to build a traceable text-retrieval pipeline for a university help desk, compare how it behaves, and explain its limitations. All documents are synthetic teaching text from the lab manual, expanded to six documents: handbook_late (215 words), handbook_late_2023 (197, an archived superseded version), it_password (191), assignment_submission (177), remote_work (177) and printer_help (181).

The query set has 8 labelled answerable queries (4 development, 4 test) and 2 unanswerable queries. Each answerable query is labelled with its document and the exact evidence text. A retrieved chunk counts as **relevant** only if it comes from the expected document **and** contains the whole evidence text, so a chunk that cuts the evidence in half is a miss.

Environment: Python 3.11.6 on CPU, numpy 2.4.6, scikit-learn 1.9.1 (TF-IDF baseline), sentence-transformers 6.2.0 with `all-MiniLM-L6-v2` (384-dimensional embeddings, max_seq_length 256). Chunk size and overlap are measured in **words**. Encode and search time were measured with `time.perf_counter()`; model loading (0.169 s) was measured separately so it does not inflate the search numbers. To reproduce: `pip install -r requirements.txt`, then `python create_dataset.py`, `python task1_tokens.py`, `python task2_chunks.py` and `python task3_retrieval.py`. Full terminal output is saved in the outputs folder.

2. Task 1 – Token comparison and FAQ chatbot

Case	Input	Whitespace	Word-like	MiniLM WordPiece
Punctuation	AI, search!	2: AI, | search!	2	4 sub-words (6 ids)
Contraction (')	I can't log in.	4 (can't kept)	5 (can | t)	7 sub-words (9 ids)
Contraction (’)	I can’t log in.	4	5 (can | t)	–
Email	help@example.com	1	3	5 sub-words (7 ids)
Course code	CS4944 version 2.1	3	4 (2 | 1)	7: cs ##49 ##44 …
Empty string	(empty)	0	0	–
Urdu	یہ ایک مثال ہے	4	4	11 sub-words (13 ids)
File name	CS4944_42365_Lab6.pdf	1	2	–

Table 1. Token counts from outputs/task1_tokens.txt. “–” means the sub-word tokenizer was not run on that input in this script.

Whitespace splitting keeps punctuation glued to words (“in.”, “search!”) and keeps an email as one token. The word-like regex removes punctuation but breaks “can't” into “can” and “t”, splits “2.1” into two numbers and destroys the email. The model tokenizer splits rare strings into pieces: CS4944 becomes cs ##49 ##44 and the four Urdu words become 11 pieces, so Urdu costs about three times more tokens per word. Aggressive normalization is risky: “CS4944 version 2.1” became “cs4944 version 21” and “help@example.com” became “helpexamplecom”, so I kept the original text separately and only normalized a search copy.

FAQ chatbot check (5 FAQs, all checks passed): “How do I reset my password?” returned the exact answer with its source; “printer shows empty paper” did not match exactly, so the bot offered the blank-pages FAQ as a suggestion (similarity 0.238) and asked the user to choose or rephrase; “wifi not working” and “volcano geology” got “No matching FAQ. Please contact the help desk.”; an empty input got “Please enter a question.” The bot only answers when the match is confirmed and otherwise falls back to suggestions or a human.

3. Task 2 – Chunking comparison and boundary tests

Three documents (587 words in total) were chunked with a 40-word budget using fixed windows without overlap, fixed windows with 10-word overlap, and a sentence-aware splitter that packs whole sentences up to 40 words.

Strategy	Chunks	Min w	Mean w	Max w	Dup. words	Labelled spans kept whole
Window 40 / overlap 0	16	15	36.7	40	0	1/3 (handbook_late)
Window 40 / overlap 10	20	11	37.9	40	170	0/3
Sentence-aware 40	21	12	28.0	40	0	1/3 (it_password)

Table 2. Chunking statistics from outputs/task2_chunks.txt.

Boundary tests (all passed): empty text, a document shorter than the budget, four invalid overlap values, a 95-word sentence that falls back to 3 word windows, and a check that every chunk's character offsets reproduce the source text exactly. The longest real sentence was 61 words, which is over the budget, so it also used the window fallback.

Overlap duplicated 170 words and still kept none of the three labelled spans whole, because the spans are longer than the 10-word overlap. The sentence splitter also showed a real weakness: it cut after the abbreviations “Dr.” and “e.g.”, producing the fragments “Dr.” and “Khan, the assessment officer, reviews medical evidence within three working days, e.g.”. In the semantic suggestion check, the paraphrase “A forgotten password can be reset using the recovery link.” scored 0.343 with TF-IDF and 0.526 with embeddings, and for “I was unwell, can I get more time for my assignment?” embeddings ranked two handbook_late sentences first and second (0.380, 0.339).

4. Task 3 – Retrieval results

Strategy	Method	Chunks	Dev Hit@3	Dev MRR@3	Test Hit@3	Test MRR@3	Encode (s)	Search (s)
Window 40/0	TF-IDF	31	0.500	0.500	0.250	0.125	0.0043	0.000208
Window 40/0	Embedding	31	0.500	0.500	0.500	0.208	0.1169	0.000062
Window 40/10	TF-IDF	39	0.250	0.250	0.250	0.125	0.0025	0.000161
Window 40/10	Embedding	39	0.250	0.125	0.250	0.125	0.1077	0.000046
Sentence-40	TF-IDF	39	0.500	0.500	0.750	0.417	0.0024	0.000167
Sentence-40	**Embedding**	39	0.500	0.500	**1.000**	**0.875**	0.0912	0.000044

Table 3. results_summary.csv (6 documents, 4 dev + 4 test answerable queries). Every chunk was at most 67 model tokens, well under the 256 limit, so nothing was truncated.

The selection rule fixed before looking at test data was: best dev MRR@3, then dev Hit@3, then fewer chunks. Four configurations tied on dev (Hit@3 0.500, MRR@3 0.500), so the tie-break on chunk count (31 against 39) left the two window 40/0 runs, and the script kept **window 40/0 + embedding**. Its held-out test score, with no retuning, was Hit@3 0.500 and MRR@3 0.208. Sentence-aware + embedding reached Hit@3 1.000 and MRR@3 0.875 on the same test queries. Embedding encoding was roughly 27–43 times slower than TF-IDF (0.09–0.12 s against 0.002–0.004 s for the whole corpus), but searching was under 0.0003 s for every configuration, so runtime is not a real problem at this size.

Unanswerable query	Config	Top-1 chunk	Score
What is the hostel fee for next semester?	Window 40/0 + embedding	handbook_late:0:0	0.294
Who won the football match yesterday?	Window 40/0 + embedding	handbook_late:0:2	0.078
What is the hostel fee for next semester?	Sentence-40 + embedding	handbook_late:sentence:0	0.336
Who won the football match yesterday?	Sentence-40 + embedding	it_password:sentence:6	0.105

Table 4. Unanswerable queries, reported separately. The system always returns three chunks, so these are false candidates.

In the interactive document chatbot, “Can contractors work remotely?” returned remote_work:0:1 (score 0.692, characters 272–517) with the exact contractor rule and its source offsets; the football question (top score 0.078) was declined by the user with Enter and the bot answered “No answer selected.

Rephrase or contact a human."; an empty question gave "Please enter a question."; and an invalid choice (9) also fell back safely. Every passage is printed as candidate evidence, not a verified answer.

5. Failure and edge-case analysis

failures.json lists 35 cases where the relevant chunk was not at rank 1 across all six configurations. Four of them explain most of what I saw:

Failure 1 – evidence cut by a window boundary (window 40/0 + embedding)

Query: "How do I reset a forgotten password?" (dev). The right document came first (it_password:0:0, score 0.554), but the evidence sentence starts in chunk 0 and continues in chunk 1 ("the link is sent to the personal email address … expires after 30 minutes"), so no chunk contains it and the result counts as a miss. The test paraphrase "How can I get back into my account if I forgot my login?" failed the same way (0.543). **Cause:** fixed windows ignore sentences. **Fix:** sentence-aware chunking, which put the full sentence in it_password:sentence:1 at rank 1 for both queries (0.756 and 0.680).

Failure 2 – archived version ranked above the current rule (sentence-40 + embedding)

Query: "What is the penalty for late work, and can illness give me an extension?" (dev). Top-1 was handbook_late_2023:sentence:2 (0.558): "Under the 2023 rules, late work was accepted for up to three days with a deduction of 5% per day…", which is out of date. The current rule is 10% per day. Also, the labelled evidence is two sentences, and with sentence chunks they ended up in different chunks, so no single chunk had both the penalty and the illness exception. **Cause:** similarity measures topic, not currency, and the question needs two sentences. **Fix:** store a version/status field and filter or down-rank archived documents; allow adjacent sentences to be merged when they belong to one rule.

Failure 3 – wrong document for a vague paraphrase (both strategies)

Query: "How should I hand in my coursework?" (test). Top-1 was the handbook_late introduction (0.451 window, 0.464 sentence) because it talks about coursework deadlines; the correct assignment_submission chunk came second (0.426 / 0.422). **Cause:** the word "coursework" and the general topic overlap between the two documents. **Fix:** add document titles to chunks, or a re-ranking step; it still counts as a Hit@3, so the user would see it.

Failure 4 – unanswerable questions still get confident-looking results

"What is the hostel fee for next semester?" returned handbook_late chunks with 0.294 (window) and 0.336 (sentence). That is higher than the top score (0.264) the chatbot gave for the real question "What proof do I need if I was sick?", so a single fixed score threshold would not separate them safely. **Fix:** a calibrated "no answer" threshold tuned on dev, plus the confirmation step the chatbot already uses. A related edge case from the guided embedding script: "I do not need a password reset" scored 0.713 against the reset sentence, higher than the real request "How can I recover my login?" (0.627), so the model largely ignores negation.

6. Chunking choice

I choose **sentence-aware chunking (40-word budget) with embedding retrieval**. Evidence for this choice: (1) on dev it tied with the best configuration; (2) it duplicates no text, while 10-word overlap added 170 duplicate words and 8 extra chunks and gave the worst dev scores (Hit@3 0.250); (3) the main failure of fixed windows was cutting evidence sentences (Failure 1), which sentence chunks avoid by design; (4) all chunks fit in the model limit. The held-out test results agree (Hit@3 1.000, MRR@3 0.875 against 0.500 and 0.208). I want to be honest that the pre-set tie-break rule selected window 40/0, and I only saw the test advantage after running it, so the test score of sentence-aware chunking is a confirmation, not an unbiased estimate. The abbreviation problem ("Dr.", "e.g.") must be fixed before using it for real.

7. Trigger questions

Does splitting at spaces correctly handle "can't", CS4944, email addresses and Urdu text?

Only partly. Spaces keep "can't", CS4944 and the email as single tokens, but they also leave punctuation attached ("in.") and give no information about the inside of a token. Urdu words are separated by spaces so the counts were the same (4), but the model tokenizer needed 11 pieces for them. Spaces are a reasonable first step, not a full tokenizer.

Why might "recover my account" retrieve a "password reset" passage without shared keywords?

Because sentence embeddings place sentences with similar meaning close together. "How can I recover my login?" scored 0.627 against "Reset a forgotten password using the recovery link." while unrelated sentences scored 0.072 and 0.018. TF-IDF can only match shared words.

What happens when a condition and its exception are placed in different chunks?

The retrieved chunk can show the rule without the exception, which gives a wrong answer. In Failure 2 the late penalty and the illness extension were separated, and in the window run the password instructions were split so the full evidence was never retrieved.

Does overlap improve evidence coverage enough to justify duplicate text and extra computation?

Not in my experiment. A 10-word overlap added 170 duplicate words and 39 instead of 31 chunks, kept 0/3 labelled spans whole, and had the lowest scores (dev Hit@3 0.250). Overlap only helps when it is longer than the evidence that crosses the boundary.

Can a high similarity score establish that a passage is correct, current or sufficient to answer?

No. The archived 2023 rule scored 0.558 and ranked first, the hostel-fee question got 0.336 with no answer in the corpus, and a negated sentence scored 0.713. A score only says the texts are similar; correctness, date and completeness have to be checked from the source.

8. Reflection

What did you learn today?

Tokens, chunks and embeddings each change what the retriever can find. Chunk boundaries mattered more than I expected: the same model went from test Hit@3 0.500 to 1.000 only by changing how text was split.

How will you apply this in a real AI system?

I would keep source IDs and character offsets on every chunk, check token limits before encoding, store document version and status, and show retrieved text as evidence with a confirmation or fallback step instead of a final answer.

What failed or surprised you?

Overlap made results worse, not better. The archived handbook ranked above the current one, the sentence splitter broke on "Dr." and "e.g.", and a negated sentence scored higher than a real paraphrase.

What would you do differently next time?

Use a larger query set (8 answerable queries means one query changes Hit@3 by 0.25), fix the abbreviation handling, add a no-answer threshold tuned on dev, and decide the tie-break rule with more care before testing.

9. Limitations

- The dataset is small and synthetic (6 documents, 8 answerable queries), so each test query is worth 0.25 Hit@3 and the differences are not statistically reliable.
- Relevance requires an exact evidence string inside one chunk. This is strict and makes split evidence a miss even when the user would still see most of it.
- Only one embedding model (all-MiniLM-L6-v2) and one chunk budget (40 words) were tested; Urdu is tokenized into many pieces and was not evaluated for retrieval.
- There is no score threshold, so unanswerable questions always return candidates; timing was measured once on one CPU and will vary between machines.
- The prototype retrieves evidence only. It must not be used for real policy or legal decisions, and the policies in it are not real university rules.